

Mini report 2

Section 1: Multiprocessing

Lecture 4

Performance table (from milestone 3 of lecture 4),

Workers	Time (s)	Speedup	Efficiency
1	0.280	0.83x	83%
2	0.172	1.35x	67%
3	0.161	1.44x	48%
4	0.124	1.86x	47%
5	0.108	2.14x	43%
6	0.103	2.24x	37%
7	0.094	2.47x	35%
8	0.091	2.55x	32%
9	0.084	2.75x	31%
10	0.080	2.89x	29%
11	0.078	2.97x	27%
12	0.070	3.29x	27%

Lecture 5

Milestone 1 result:

When used `n_chunks = n_worker * 4`

Workers	Time (s)	Speedup	Efficiency
---------	----------	---------	------------

1	0.363	0.35	35%
2	0.182	0.69	34%
3	0.131	0.96	32%
4	0.110	1.14	29%
5	0.100	1.26	25%
6	0.120	1.04	17%
7	0.103	1.22	17%
8	0.096	1.31	16%
9	0.097	1.29	14%
10	0.091	1.38	14%
11	0.084	1.49	14%
12	0.085	1.47	12%

Best: $n_workers = 11$ — highest speedup at 1.49× when $n_chunks = 4 * n_workers$.

Milestone 2:

For milestone 2, I have to find out LIF (load imbalance factor). From the lecture 4 benchmarking table, it can be seen that when $n_workers = 12$, it gives the highest speed up with the same efficiency as $n_workers = 11$. That is why my best **$n_workers = 12$** .

Result:

$n_workers = 12$

n_chunks	time (s)	vs. 1x	LIF
$1 \times n_workers$	0.5704	1.0000	11.0000
$2 \times n_workers$	0.5370	1.0623	10.2962
$4 \times n_workers$	0.5388	1.0587	10.3351
$8 \times n_workers$	0.5936	0.9611	11.4863
$16 \times n_workers$	0.5586	1.0213	10.7500
$32 \times n_workers$	0.7380	0.7730	14.5244

For me, $n_chunks = 2 * n_workers$ is the best LIF as it is the lowest. Also, when I increase the chunk size to $32 * n_workers$, the LIF jumps up, which is inefficient.

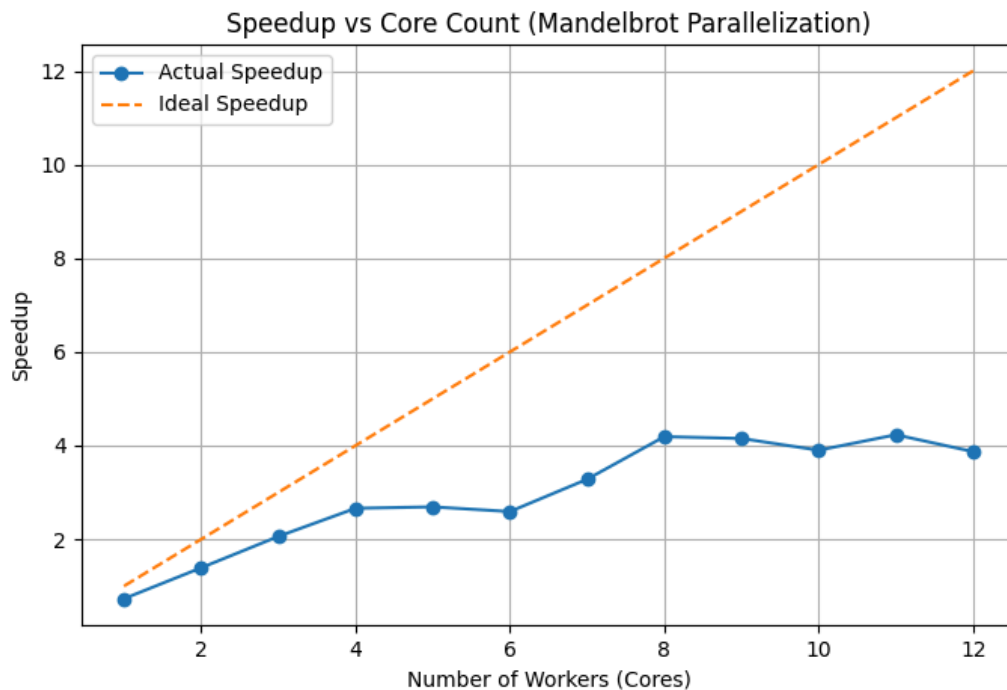
Milestone 3:

1. Speedup table:

Implementation	Time (s)	Speedup
Naive Python	3.173	1×
NumPy	0.536	5.93×
Hybrid	1.097	2.89×
Numba (@njit)	0.045	70.32×
Parallel (opt.)	0.101	31.51×

2. Speedup vs. core count:

Workers	Time (s)	Speedup	Efficiency	s (Serial Fraction)
1	0.173	0.73	0.73	0.000
2	0.091	1.39	0.69	0.441
3	0.061	2.06	0.69	0.229
4	0.047	2.66	0.67	0.167
5	0.047	2.69	0.54	0.215
6	0.049	2.59	0.43	0.263
7	0.038	3.28	0.47	0.189
8	0.030	4.19	0.52	0.130
9	0.030	4.15	0.46	0.146
10	0.032	3.90	0.39	0.173
11	0.030	4.23	0.38	0.160
12	0.033	3.87	0.32	0.191



From the graph and table, it can be seen that when the number of cores is 8, 13% of the program's code is serialized. After that, even with more cores, serialization increases.

3. Brief recommendation:

What settings give the best time?

-> When the number of cores is 8, 9, and 11, the task finished in 0.030 seconds, which is the lowest. So, `n_cores = 8, 9, or 11` gives the best time, but if we consider other factors like efficiency and serial fraction, then `n_cores = 8` is the winner.

Is parallelization worth it on your hardware?

-> Yes, if we look at the speedup vs. core count table, we can see that up to 8 cores, the computation time decreases. After that, the speedup does not improve. So, it is worth it.

Write 2–3 sentences in your performance notebook

-> Increasing the number of workers does not always improve speedup, because not every part of the program can be parallelized. Even with the Mandelbrot problem, where most of the computation can be done in parallel, program serialization began to increase after 8 cores.

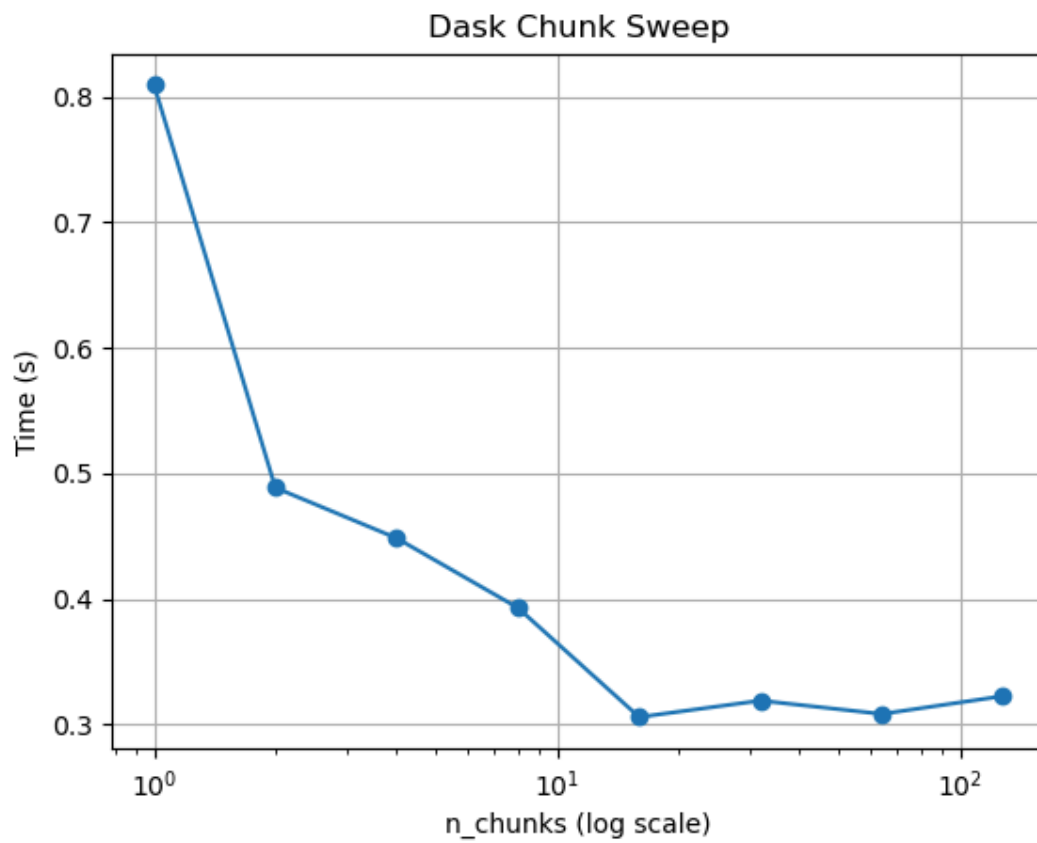
Section 2: Dask local

Lecture 6:

Milestone 1:

Implementation	Median Time (s)	N	Chunks	Workers
Dask (delayed, Numba chunks)	0.3058	4096	16	8

Chunk sweep:



n_chunks	Time (s)	vs 1×	Speedup	LIF
1	0.8094	1.000	1.000	7.000
2	0.4886	0.604	1.657	3.829
4	0.4483	0.554	1.805	3.431
8	0.3929	0.485	2.060	2.883
16	0.3058	0.378	2.647	2.023 ← best
32	0.3189	0.394	2.538	2.152
64	0.3083	0.381	2.625	2.047
128	0.3225	0.398	2.510	2.188

SpeedUp:

Calculated in 4096 resolution

Implementation	Time (s)	Speedup vs Naive
Naive Python	74.508	1×
NumPy	17.787	4.19×
Hybrid	27.010	2.76×
Numba (@njit)	0.858	86.86×
Numba + Parallel	1.162	64.13×
Dask local	0.866	86.08×

How does Dask local compare to multiprocessing at the same worker count?

-> When the number of workers is 8 for both multiprocessing and dask local, the dask local performs far better. Multiprocessing takes 1.162 seconds; dask local takes 0.866 seconds at 4096 resolution

What does the overhead difference tell you about when to choose each tool?

-> The numba njit is the best among all. Even in 4096 resolution. The reason might be that there is not much overhead in numba njit compared to the multiprocessing and Dask local. But if the resolution is increased from 4096, Dask local might perform better than other

methods. So, if the problem is bigger, I would choose Dask local, but if the problem is small, I would choose numba njit.

Section 3: Dask Distributed

Lecture 7:

Chunk sweep:

n_chunks	Time (s)	vs 1×	Speedup
1	1.4199	1.000	1.000×
2	1.1112	0.783	1.278×
4	0.9775	0.688	1.453×
8	0.7269	0.512	1.953×
16	0.7379	0.520	1.924×
32	0.5384	0.379	2.637× ← best
64	0.5481	0.386	2.590×
128	0.5921	0.417	2.398×

Worker scaling:

The worker scaling is computed using a 1024 resolution for faster computation. This could be the reason for the significant efficiency drop at a worker count of 3, due to higher overhead compared to choosing a 4096 resolution.

Workers	Configuration	Time (s)	Speedup (T_1/T_p)	Efficiency (Speedup / p)
1	Dask strato (n_chunks = 32)	0.133	1.00	1.00
2	Dask strato (n_chunks = 32)	0.094	1.41	0.71

3	Dask strato (n_chunks = 32)	0.084	1.58	0.53
---	-----------------------------	-------	------	------

Speed Up:

Calculated on 4096 resolution

Implementation	Time (s)	Speedup vs Naive
Naive Python	88.776	1×
NumPy	18.237	4.87×
Hybrid	29.740	2.99×
Numba (@njit)	1.243	71.44×
Numba + Parallel	3.997	22.21×
Dask local	1.305	68.00×
Dask strato	1.377	64.48×

Performance table for all implementations:

Implementation	1024×1024 Time (s)	1024 Speedup	4096×4096 Time (s)	4096 Speedup
Naive Python	5.598	1×	88.776	1×
NumPy	0.586	9.55×	18.237	4.87×
Hybrid	1.858	3.01×	29.740	2.99×
Numba (@njit)	0.075	74.26×	1.243	71.44×
Numba + Parallel	0.383	14.62×	3.997	22.21×
Dask local	0.106	52.58×	1.305	68.00×
Dask strato	0.099	56.82×	1.377	64.48×

I also tried to run a benchmark at a resolution higher than 4096, but the task was being killed by the VMs. From the table above, as resolution increases, Numba njit's speedup does not increase; instead, it decreases. But for Dask local and cluster, as the resolution increases,

their speedup also increases. From this, it can be said that a resolution greater than 4096 will balance the trade-off between overhead and speed-up in Dask's solutions.